SRE / DevOps Intern

Take-Home Technical Assessment

Monitoring • Infrastructure Automation • Troubleshooting

Candidate Name **GitHub Repo URL**	
Deadline	72 hours from receipt of this document
Submission	Push to a public GitHub repo — share the link

How This Assessment Works

There are three technical tests. Each one has a clear objective and tells you what we expect you to deliver. Read each test fully before you start.

#	Test	Type	What You Submit
1	Monitoring Stack — Choose Your Tools	Hands-on setup + documentation	Config files + dashboards + README
2	Infrastructure Automation	Hands-on setup + documentation	IaC code + README
3	Troubleshooting Scenarios	Written answers	Markdown files in repo

★ **Azure Knowledge**	Knowledge of Azure (AKS, VMs, NSGs, ACR, etc.) is a strong advantage. Where you use or reference Azure-specific approaches, make that clear in your documentation.

✓ **What We Value**	Clean, readable work — not perfection Your reasoning — tell us WHY you made your choices, not just HOW Honesty — if something didn't work, document it and explain what you tried

Repository Structure

Organise your public GitHub repository as follows. Keep folders clean — one folder per test.

Path	Contents
test-1-monitoring/	All files for Test 1
test-1-monitoring/config/	Tool config files (Promtail, Loki, Prometheus, etc.)
test-1-monitoring/dashboards/	Exported Grafana dashboard JSON files
test-1-monitoring/alerts/	Alert rule exports or screenshots
test-1-monitoring/README.md	Test 1 documentation (required)
test-2-automation/	All files for Test 2
test-2-automation/terraform/	Terraform .tf files (if used)
test-2-automation/ansible/	Ansible files (if used)
test-2-automation/README.md	Test 2 documentation (required)
test-3-troubleshooting/	All files for Test 3
test-3-troubleshooting/scenario-1.md	Answer to Scenario 1
README.md	Top-level repo overview

Test 1 — Monitoring Stack: Choose Your Tools

Objective

You are joining a team that runs applications on an AKS (Azure Kubernetes Service) cluster. The team currently has no monitoring in place. You have been asked to set up a logging and metrics monitoring solution for the cluster.

The catch: we have not told you which tools to use. Part of this test is your ability to choose the right tools, justify that choice, and then set them up correctly.

Part A — Tool Selection & Justification

Before you set anything up, write a short Tool Selection document in your test-1-monitoring/README.md. Answer these questions:

- **Which logging tools did you choose?** (e.g. Promtail + Loki + Grafana, Fluentd + Elasticsearch + Kibana, or another stack)
- **Why did you choose them?** Consider: ease of setup, resource usage, AKS compatibility, cost, and what the team will need to maintain
- **Which metrics tools did you choose?** (e.g. Prometheus + Grafana, Azure Monitor, Datadog, or another approach)

Part B — Set Up the Monitoring Stack

Set up your chosen monitoring stack. It can be on any environment you have access to — a local Kubernetes cluster (e.g. using kind or minikube), an AKS cluster if you have Azure access, or even Docker Compose to simulate the stack. Document what environment you used.

Logging Setup

- Deploy your log shipper (e.g. Promtail as a DaemonSet or equivalent) so it collects logs from all running pods/containers
- Deploy your log aggregator (e.g. Loki) to receive and store those logs
- Connect Grafana (or your chosen visualisation tool) to the log aggregator as a data source
- Verify you can see container logs in the UI — include a screenshot

Metrics Setup

- Verify you can query cluster metrics (CPU, memory, pod count) — include a screenshot

Part C — Dashboards & Alerts

This is the most important part of Test 1. We want dashboards that are actually useful, not just default imports.

Build these dashboards in Grafana

- **Dashboard 1 — Cluster Health Overview:** show CPU usage, memory usage, number of running pods, and number of failed pods across the cluster
- **Dashboard 2 — Application Logs:** show logs from all pods, filterable by namespace and pod name. Include a panel showing error-level log count over time
- **Dashboard 3 — Your Choice:** build one more dashboard that you think would be most useful to an on-call engineer. Explain in the README why you chose it

Set up these alerts

- **Alert 1:** A pod has been in CrashLoopBackOff state for more than 5 minutes
- **Alert 2:** Node CPU usage exceeds 80% for more than 3 minutes
- **Alert 3:** Your choice — add one alert you think is important for this kind of cluster. Explain why

What to Submit for Test 1

- All config files (DaemonSet YAMLs, Prometheus config, Loki config, Promtail config) in test-1-monitoring/config/
- Screenshots folder with at least 3 screenshots showing the stack working
- A clear README.md covering: tool selection rationale, setup steps, dashboard descriptions, alert descriptions, and what you would improve

Test 2 — Infrastructure Automation

Objective

The team wants to stop clicking through the Azure Portal to create infrastructure. You have been asked to automate the provisioning of a simple cloud environment using Infrastructure as Code (IaC) tools.

Like Test 1, you have a choice of tools. Terraform is the primary IaC tool we use, but you can supplement it with Ansible, Bash scripts, or any other tool that helps. Tell us what you chose and why.

Part A — Tool Choice & Justification

In your test-2-automation/README.md, answer these questions before showing any code:

- **Which IaC tool(s) did you use?** (e.g. Terraform only, Terraform + Ansible, Terraform + Bash, Pulumi, etc.)
- **Why did you combine them that way — or why did you stick to one tool?** What does each tool do best in your setup?
- **How does your approach handle secrets and sensitive values?** (e.g. Terraform variables marked sensitive, Ansible Vault, environment variables)

Part B — What to Provision

Provision the following infrastructure. You can target Azure (preferred), AWS, GCP, or a local environment like Vagrant/VirtualBox if you do not have cloud access. Document what you used.

Required Resources

- A Virtual Network (VNet on Azure, VPC on AWS, or equivalent) with at least two subnets
- Two Virtual Machines — Ubuntu 22.04:
  - VM 1: a publicly accessible VM (this represents the gateway)
  - VM 2: an internal VM with no public IP (this represents an app server)
- A firewall rule / NSG / security group that:
  - Allows SSH (port 22) to VM 1 from your IP only
  - Allows HTTP (port 80) and HTTPS (port 443) to VM 1 from anywhere
  - Allows all traffic between VM 1 and VM 2 within the private subnet
  - Denies all other inbound traffic to VM 2

If You Use a Configuration Tool (Ansible or similar)

After the VMs are provisioned, use your chosen configuration tool to do at least one of the following on VM 1:

- Install and start nginx or apache2
- Configure the system hostname
- Create a user and set up SSH key access
- Or something else useful — document what you did and why

Tip	Terraform provisions the infrastructure. Ansible (or another tool) configures what runs on it. Using both together is the real-world pattern — but you don't have to. Explain your choice.

What to Submit for Test 2

- All Terraform files in test-2-automation/terraform/
- Any Ansible playbooks or other config files in their respective folders
- Output of terraform plan saved as plan-output.txt (required even if you could not apply)
- Screenshots of provisioned resources if you ran a live deployment
- A clear README.md covering: tool choice rationale, steps to run the code, what it provisions, and what you would add in production

Test 3 — Troubleshooting Scenarios

Objective

This test has no setup required. Answer each scenario as a markdown file in your test-3-troubleshooting/ folder. We want to see how you think through a problem — not just what the answer is.

How to Answer Each Scenario	1. What do you think the most likely root cause is? 2. What steps / commands would you run to investigate? 3. How would you fix it? Keep answers concise — bullet points are fine. Honest reasoning beats a confident wrong answer.

Scenario 1 — Pods Running But Application is Unreachable (scenario-1.md)

You have deployed an application to AKS. The pods are all showing Running in kubectl get pods. But when you try to reach the application via its external URL, you get a connection timeout. No recent changes were made to the application code.

1. What are the first 3 kubectl commands you run to start investigating?
2. Which Kubernetes resources would you check first — Deployment, Service, Ingress, or NSG? Walk through your reasoning.
3. How would you test whether the problem is at the pod level, the service level, or the ingress/network level?
4. Name two Azure-specific things that could cause this even when the Kubernetes resources all look correct.

Submission Checklist

Before you share your repo link, go through this checklist.

✓	Item
Test 1	Config files for your monitoring stack are in test-1-monitoring/config/
Test 1	At least 3 screenshots showing the monitoring stack working
Test 1	README.md explains tool choices, setup steps, and dashboard decisions
Test 2	Terraform files are in test-2-automation/terraform/
Test 2	Any Ansible or other config tool files are included
Test 2	README.md explains why you chose the tools and how to run the code
Test 3	scenario-1.md answered
All	Repository is public and the link works
All	No passwords, API keys, or credentials are committed to the repo
All	Top-level README.md gives a brief overview of the whole submission

⚠ Important	Do not commit secrets. Use placeholder values or environment variables for anything sensitive. If your repo is private, the reviewer will not be able to see it — make sure it is public.

? Questions?	If anything in this assessment is unclear, reach out to your recruiter. If you make an assumption, document it in the relevant README — that is the right approach.